Énoncé du TP : Implémentation d'un système de messagerie distribuée

Contexte du travail

Ce TP a pour objectif de développer un système de messagerie en **Java**. Vous serez amené à implémenter progressivement des protocoles de communication (SMTP et POP3), gérer des accès concurrents, introduire des fonctionnalités avancées comme l'authentification via RPC et la persistance des données dans une base de données, et enfin améliorer la scalabilité et la résilience du système grâce à des architectures modernes.

Ce TP est conçu pour vous guider dans la création progressive d'un système de messagerie distribué, tout en explorant des concepts clés des systèmes distribués comme les protocoles, le parallélisme, les API RESTful, et la scalabilité. Chaque étape constitue une brique essentielle pour comprendre les défis et proposer des solutions pour des systèmes distribués modernes.

Remarque 1 : Chaque étape du développement doit être versionnée et documentée dans un dépôt Git. Vous devez donc installer et configurer un client Git pour committer (commit) et pousser (push) vos modifications après chaque fonctionnalité ou correction.

Remarque 2 : Un rapport doit être rédigé en plusieurs sections, correspondant aux différentes parties du TP. Après chaque phase de développement et de test, vous devez commenter les développements effectués ainsi que les résultats obtenus.

Objectifs du TP :

1. Comprendre et implémenter les protocoles SMTP et POP3 pour un système de messagerie ;
2. Explorer les concepts de parallélisme dans les systèmes distribués ;
3. Manipuler différentes technologies de stockage (fichiers locaux, bases de données relationnelles) ;
4. Comprendre et implémenter les appels de procédures à distance (RPC) avec Java RMI ;
5. Concevoir et utiliser des API RESTful pour exposer des services de messagerie ;
6. Enfin, appréhender les concepts avancés de scalabilité, de tolérance aux pannes et de load balancing dans des environnements distribués.

1 Implémentation des Protocoles SMTP et POP3 avec les Sockets

1.1 Objectif :

Implémenter les protocoles SMTP et POP3 en utilisant **les sockets** et en respectant les spécifications des RFC correspondants (**RFC 5321** [1] **pour SMTP** et **RFC 1939** [2] **pour POP3**) voir Figure 1.

1.2 Téléchargement et lecture des RFC :

Veuillez-vous référer aux RFC 5321 (SMTP) et RFC 1939 (POP3) pour comprendre le fonctionnement des commandes des deux protocoles. Ces documents détaillent les spécifications techniques nécessaires à l'implémentation des protocoles.

1. https ://www.rfc-editor.org/info/rfc5321
2. (https ://www.rfc-editor.org/info/rfc1939)

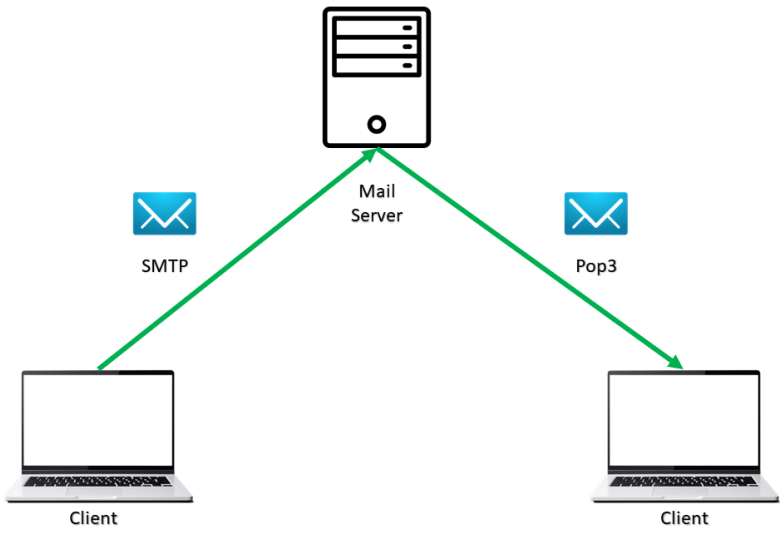

FIGURE 1 – Architecture globale du serveur de messagerie

1.3 Respect des normes du protocole via un automate à états finis :

Afin de garantir le bon déroulement des commandes et le respect des spécifications des protocoles SMTP et POP3, vous devez implémenter un automate à états finis pour gérer les transitions entre les différents états de communication. Certaines commandes ne sont autorisées que dans des états spécifiques (par exemple, la commande DATA ne peut être exécutée qu'après MAIL FROM et RCPT TO en SMTP). L'automate doit valider chaque commande en fonction de l'état courant du protocole et rejeter les commandes non autorisées avec le code d'erreur approprié.

1.4 Développement du serveur SMTP :

Le serveur SMTP qui implémente le protocole SMTP doit gérer les commandes de base (**HELO, MAIL FROM, RCPT TO, DATA, QUIT, etc.**) et répondre avec les codes appropriés par exemple : 220 pour la connexion, 250 pour les commandes standards, 354 pour **DATA** et 221 pour **QUIT**.

Le stockage des emails s'effectue dans une structure de répertoires organisée comme suit : un dossier principal "**mailserver/**" contenant des sous-dossiers pour chaque utilisateur "**mailserver/user/**" voir Figure 2. Chaque email sera stocké dans un fichier sous le nom **timestamp.txt** en respectant la norme du format des emails défini par RFC 5322 [3].

Dans cette étape, pour des raisons de simplicité, veuillez répondre positivement à la commande `MAIL FROM` ainsi qu'à la commande `RCPT TO` si les répertoires associés existent. En revanche, si les répertoires n'existent pas, veuillez répondre négativement.

1.4.1 Tests du Protocole SMTP avec Différents Scénarios

Valider le bon fonctionnement du serveur SMTP implémenté en testant différents scénarios de communication. Ces tests permettront de vérifier que le serveur respecte les spécifications du protocole SMTP (RFC 5321) et qu'il gère correctement les commandes, les erreurs et les transitions d'état.

Scénarios de test à implémenter

1. **Scénario de base :**
   - Connexion au serveur SMTP.
   - Envoi d'un email simple avec les commandes `HELO`, `MAIL FROM`, `RCPT TO`, `DATA` et `QUIT`.

3. https ://www.rfc-editor.org/info/rfc5322

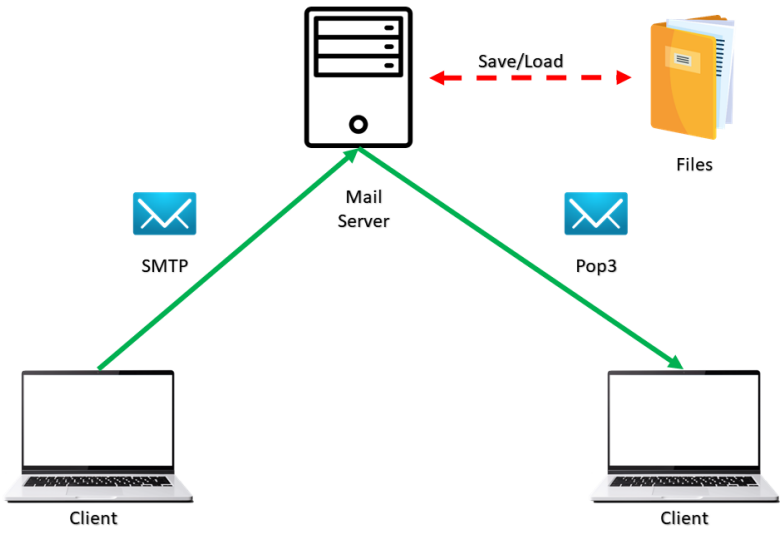

FIGURE 2 – Implémentation des Protocoles SMTP et POP3 avec des Sockets

- Vérifier que le serveur répond avec les codes appropriés (220, 250, 354, 221, etc) et que l'email est correctement stocké dans le dossier de l'utilisateur.

2. **Scénario avec plusieurs destinataires :**
   - Envoyer un email à plusieurs destinataires en utilisant plusieurs commandes `RCPT TO`.
   - Vérifier que l'email est correctement distribué à tous les destinataires à travers la création des fichiers correspondants.
3. **Scénario avec commandes dans le mauvais ordre :**
   - Essayer d'envoyer des commandes dans un ordre incorrect (par exemple, `DATA` avant `MAIL FROM` ou `RCPT TO`).
   - Vérifier que le serveur rejette les commandes non autorisées avec le code d'erreur approprié (par exemple, 503 pour « Bad sequence of commands »).
4. **Scénario avec interruption de la connexion :**
   - Interrompre la connexion pendant l'envoi d'un email (par exemple, fermer la connexion après `DATA` mais avant la fin du message).
   - Vérifier que le serveur gère correctement l'interruption et ne stocke pas l'email incomplet.
5. **Scénario avec email volumineux :**
   - Envoyer un email avec un corps de message volumineux (par exemple, plusieurs Mo).
   - Vérifier que le serveur gère correctement les emails volumineux et que le message est stocké sans erreur.
6. **Scénario avec erreurs de syntaxe :**
   - Envoyer des commandes avec des erreurs de syntaxe (par exemple, `MAIL FROM` sans adresse email valide).
   - Vérifier que le serveur répond avec le code d'erreur approprié (par exemple, 501 pour « Syntax error in parameters or arguments »).

Outils de test

- Utilisez **Telnet** ou un client SMTP personnalisé pour simuler les scénarios de test.
- Vous pouvez également utiliser des outils comme Microsoft Ouotlook pour envoyer des emails.

Discussion

- Pour chaque scénario, documentez les réponses du serveur et vérifiez que le comportement est conforme aux spécifications du protocole.
- Fournissez des captures d'écran ou des logs des tests pour valider les résultats.
- Analysez et commentez les résultats obtenus dans votre rapport. Discutez des écarts éventuels par rapport aux attentes, des problèmes rencontrés et des améliorations possibles.

1.5 Développement du serveur POP3 :

Le serveur POP3 qui implémente le protocole POP3 doit gérer les commandes de base (**USER, PASS, STAT, LIST, RETR, DELE, NOOP, RSET, QUIT**) et répondre avec les codes appropriés, par exemple :

+OK pour une réponse valide.

-ERR pour une erreur ou une commande invalide.

Les emails sont stockés dans le dossier principal "**mailserver/**" contenant des sous-dossiers pour chaque utilisateur "**mailserver/user/**".

Dans cette étape, pour des raisons de simplicité, veuillez répondre à la commande `USER` par `+OK` si le répertoire associé existe, et par `-ERR` dans le cas contraire. En ce qui concerne la commande `PASS`, répondez simplement par `+OK`.

1.5.1 Tests du Protocole POP3 avec Différents Scénarios

Valider le bon fonctionnement du serveur POP3 implémenté en testant différents scénarios d'interaction. Ces tests permettront de vérifier que le serveur respecte les spécifications du protocole POP3 (RFC 1939) et qu'il gère correctement les commandes, les erreurs et les transitions d'état.

Scénarios de test à implémenter

1. **Scénario de base :**
   - Connexion au serveur POP3.
   - Récupération de la liste des emails avec `STAT` et `LIST`.
   - Lecture d'un email avec `RETR`.
   - Déconnexion avec `QUIT`.
   - Vérifier que le serveur répond avec les codes appropriés (`+OK`, `-ERR`).
2. **Scénario avec suppression de messages :**
   - Supprimer un email avec `DELE`.
   - Se reconnecter et vérifier que le message supprimé n'est plus disponible après `QUIT`.
3. **Scénario avec annulation des suppressions :**
   - Supprimer un email avec `DELE`.
   - Annuler la suppression avec `RSET`.
   - Vérifier que le message est toujours disponible après la reconnexion.
4. **Scénario avec accès à un message inexistant :**
   - Tenter d'accéder à un message qui n'existe pas (`RETR 999`).
   - Vérifier que le serveur renvoie une erreur `-ERR No such message`.
5. **Scénario avec connexion interrompue :**
   - Interrompre la connexion après l'authentification sans exécuter `QUIT`.
   - Vérifier que les suppressions marquées ne sont pas appliquées.
6. **Scénario avec une boîte aux lettres volumineuse :**
   - Tester un utilisateur ayant un grand nombre d'emails.
   - Vérifier que les commandes `STAT`, `LIST` et `RETR` fonctionnent correctement sans ralentissements excessifs.

Outils de test

- Utilisez **Telnet** ou un client POP3 personnalisé pour simuler les scénarios de test.
- Vous pouvez également utiliser des clients de messagerie comme Thunderbird ou Outlook pour tester la récupération des emails.

Discussion

- Pour chaque scénario, documentez les réponses du serveur et vérifiez que le comportement est conforme aux spécifications du protocole.
- Fournissez des captures d'écran ou des logs des tests pour valider les résultats.
- Analysez et commentez les résultats obtenus dans votre rapport. Discutez des écarts éventuels par rapport aux attentes, des problèmes rencontrés et des améliorations possibles.

2 Problème d'accès simultané et implémentation du parallélisme :

Dans le contexte d'un serveur de messagerie, il est essentiel de gérer les accès simultanés de plusieurs clients. En effet, plusieurs clients peuvent tenter d'envoyer des emails via le serveur SMTP en même temps, tandis que d'autres peuvent chercher à lire leurs messages à travers le serveur POP3. Cela crée des situations où des écritures et des lectures peuvent se produire simultanément sur les mêmes ressources, entraînant des problèmes potentiels d'intégrité des données et d'état des sessions.

Pour répondre à ce défi, il est nécessaire aussi de mettre en oeuvre une gestion des accès concurrents. Chaque client doit voir son état enregistré de manière à éviter toute interférence entre les connexions. Cela implique l'utilisation de mécanismes de synchronisation qui garantissent que les opérations d'écriture (envoi d'emails) et de lecture (récupération d'emails) peuvent se produire sans conflit.

L'implémentation du parallélisme peut être réalisée grâce à l'utilisation de **threads** en Java. Chaque connexion client sera traitée dans un thread distinct, permettant ainsi l'exécution parallèle des opérations.

2.1 Scénarios de test à implémenter :

1. **Scénario de charge simultanée :**
   - Simuler plusieurs clients envoyant des emails via le serveur SMTP et récupérant des emails via le serveur POP3 en parallèle.
2. **Scénario de traitement parallèle des messages :**
   - Lorsqu'un client envoie un email, initier simultanément la récupération des emails pour un autre utilisateur dans un thread séparé.
   - Vérifier que les deux opérations (envoi et réception) s'exécutent sans interférence et que les messages sont traités correctement.
3. **Scénario de saturation de la file d'attente :**
   - Tester le comportement du système lorsque le nombre de connexions dépasse la capacité maximale du pool de threads.
   - Évaluer comment le système gère les connexions en attente et s'assure que les requêtes ne sont pas perdues ou abandonnées tout en maintenant des threads actifs pour d'autres connexions.

3 Serveur d'Authentification avec Java RMI

L'objectif de cette partie est d'utiliser Java RMI pour gérer les comptes utilisateurs à distance. Les serveurs SMTP et POP3 doivent invoquer des procédures distantes pour vérifier la validité des utilisateurs. De plus, un client RMI sera développé pour gérer les comptes utilisateurs.

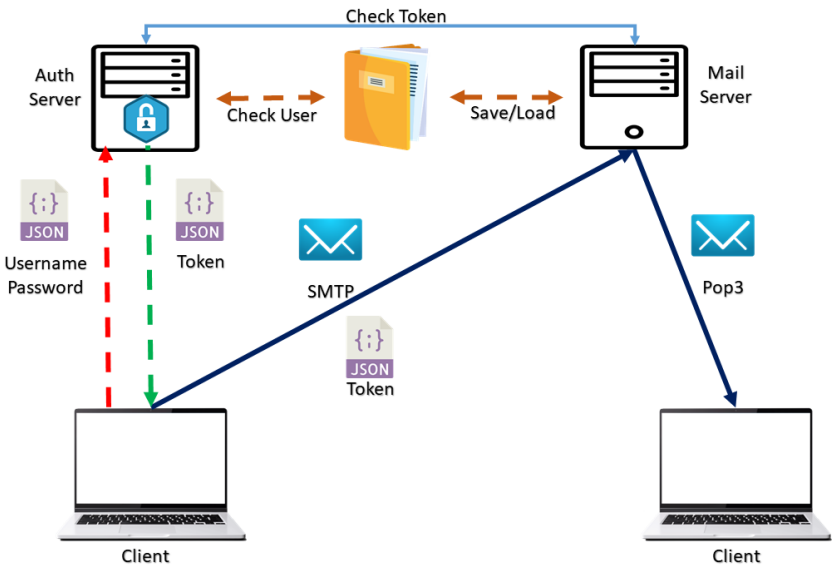

FIGURE 3 – Serveur d'Authentification avec Java RMI

3.1 Mise en place d'un serveur RMI

Le serveur d'authentification doit être conçu comme un service Java RMI qui offre les fonctionnalités suivantes :

— Vérification des identifiants et mots de passe des utilisateurs.

— Création de nouveaux comptes utilisateurs.

— Mise à jour et suppression de comptes existants.

3.2 Stockage local des comptes

Les comptes utilisateurs doivent être stockés dans un fichier au format JSON ou XML. Le serveur RMI doit lire et écrire dans ce fichier afin de conserver l'état des comptes de manière persistante.

3.3 Intégration avec les serveurs SMTP et POP3

Les serveurs SMTP et POP3 doivent être modifiés pour interagir avec le serveur RMI afin d'authentifier les utilisateurs de manière centralisée.

Lorsqu'un client POP3 tente de se connecter, son identifiant et son mot de passe sont transmis au serveur RMI pour vérification. De même, avant d'accepter l'envoi d'un email, le serveur SMTP interroge le serveur RMI afin de s'assurer que l'expéditeur est un utilisateur valide.

3.4 Développement d'un client Java RMI

Un client Java RMI doit être développé pour permettre la gestion des comptes utilisateurs. Il devra inclure une interface graphique intuitive permettant d'ajouter, de mettre à jour et de supprimer des comptes. Ce client devra interagir avec le serveur Java RMI en effectuant des requêtes à distance pour la gestion des comptes de manière centralisée et sécurisée.

3.5 Tests et validation

Afin de valider l'intégration du serveur d'authentification Java RMI avec les serveurs SMTP et POP3, plusieurs scénarios de test doivent être réalisés :

— Test de création de compte : Vérifier que le client RMI permet d'ajouter un nouvel utilisateur et que ce dernier est bien enregistré dans le fichier JSON/XML.

— Test de suppression de compte : Supprimer un utilisateur et s'assurer qu'il n'est plus reconnu par les serveurs SMTP et POP3.

— Test d'authentification réussie : Tenter une connexion avec des identifiants valides et vérifier que l'accès est accordé.

— Test d'authentification échouée : Tester avec un mot de passe incorrect et s'assurer que l'accès est refusé.

— Test de vérification des utilisateurs par le serveur SMTP : Vérifier que le serveur SMTP consulte correctement le serveur d'authentification.

3.5.1 Discussion

— Pour chaque scénario, documentez les réponses du serveur et vérifiez que le comportement est conforme aux spécifications du protocole.

— Fournissez des captures d'écran ou des logs des tests pour valider les résultats.

— Analysez et commentez les résultats obtenus dans votre rapport. Discutez les problèmes rencontrés et proposez des améliorations possibles.

4 Intégration d'une Base de Données (MySQL)

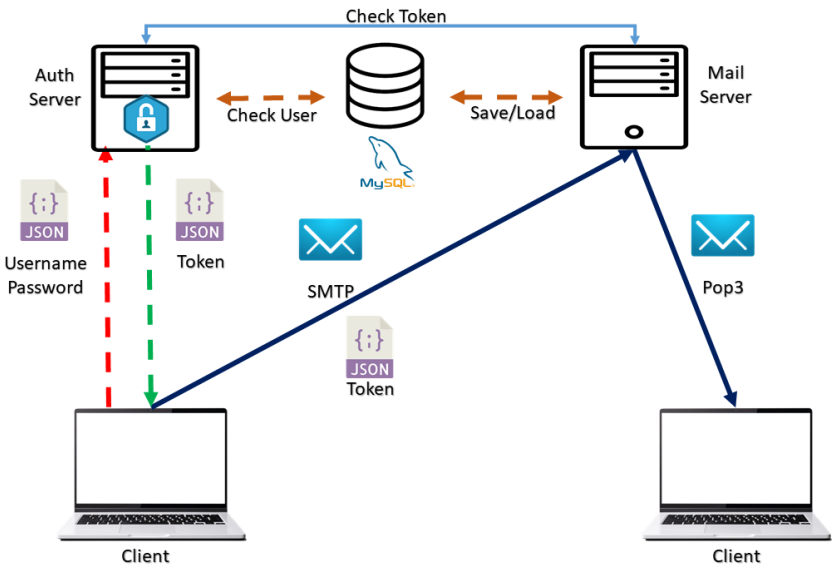

FIGURE 4 – Intégration d'une Base de Données (MySQL)

À partir de cette partie, le système de fichiers utilisé pour stocker les emails et les comptes utilisateurs sera remplacé par une base de données MySQL. Cette approche permet :

— Une **gestion centralisée** des utilisateurs et des emails.

— Une **meilleure cohérence des données** et une **sécurité renforcée**.

— Une **scalabilité améliorée**, adaptée à un grand volume d'emails et de comptes.

Le serveur **SMTP** stockera désormais les emails dans la base de données, tandis que le serveur **POP3** les récupérera à partir de cette base. De même, le **serveur d'authentification** gérera les comptes utilisateurs directement en base de données.

4.1 Conception de la Base de Données

La base de données contiendra au minimum deux tables principales :

— **users** : Stocke les informations des utilisateurs, y compris l'identifiant, le mot de passe (haché et en claire) et éventuellement d'autres métadonnées (date de création, statut, etc.).

— **emails** : Contient les emails envoyés et reçus, avec les métadonnées nécessaires (expéditeur, destinataire, sujet, date d'envoi, état de lecture, contenu, etc.).

Dans le rapport final, vous devez :

— Présenter le **schéma conceptuel** de la base de données sous forme d'un diagramme UML (Modèle Entité-Association).

— Justifier les choix de conception effectués.

4.2 Implémentation des Procédures Stockées

Pour optimiser les interactions entre les serveurs et la base de données, des procédures stockées seront mises en place (à titre d'exemple) :

— `authenticate_user(username, password)` : Vérifie l'identité d'un utilisateur.

— `store_email(sender, recipient, subject, content)` : Insère un nouvel email.

— `fetch_emails(user)` : Récupère la liste des emails d'un utilisateur donné.

— `delete_email(email_id)` : Supprime un email spécifique.

— `update_password(user, new_password)` : Modifie le mot de passe d'un utilisateur.

4.3 Intégration avec les Serveurs

Les serveurs devront être adaptés pour interagir avec MySQL, par exemple :

— Le serveur **SMTP** doit appeler la procédure `store_email()` lors de l'envoi d'un email.

— Le serveur **POP3** doit exécuter `fetch_emails()` et `delete_email()` pour récupérer ou supprimer des emails.

— Le **serveur d'authentification** remplacera le stockage XML/JSON par la table `users` et la procédure `authenticate_user()`.

L'accès à la base de données se fera via JDBC pour assurer la communication entre les serveurs Java et MySQL.

4.4 Tests et Validation

Une série de tests sera effectuée pour vérifier le bon fonctionnement de l'intégration :

— **Tests fonctionnels** :
  — Création et authentification d'utilisateurs via le serveur d'authentification.
  — Envoi d'un email via le serveur SMTP et vérification de son insertion en base.
  — Récupération et suppression d'un email via le serveur POP3.

— **Tests de performance** :
  — Évaluation des temps d'accès et de traitement pour un grand volume d'emails.
  — Tests de charge avec plusieurs utilisateurs simultanés.

— **Tests de robustesse et de sécurité** :
  — Tentatives d'authentification avec des identifiants erronés.
  — Vérification des requêtes SQL pour éviter les injections SQL.

4.5 Discussion

— Documentez les réponses du serveur et vérifiez que le comportement est conforme aux spécifications du protocole.

— Fournissez des captures d'écran ou des logs des tests pour valider les résultats.

— Analysez et commentez les résultats obtenus dans votre rapport. Discutez les problèmes rencontrés et proposez des améliorations possibles.

5 Intégration d'un Serveur RESTful

L'objectif de cette partie est d'exposer les fonctionnalités du serveur via une API REST afin de permettre l'accès à distance aux services d'authentification, d'envoi et de consultation des emails. L'intégration d'un serveur RESTful apporte plusieurs avantages :

- **Interopérabilité** : Permet à n'importe quel client (application web, mobile, autre serveur) d'accéder aux services via des requêtes HTTP standard.
- **Scalabilité** : Facilite la montée en charge en permettant l'intégration avec d'autres services et applications.
- **Séparation des responsabilités** : Le serveur RESTful agit comme une couche d'abstraction entre les clients et les serveurs SMTP, POP3 et d'authentification.
- **Facilité de test et de maintenance** : Les endpoints REST permettent une validation rapide avec des outils comme **Postman** ou **cURL**.

5.1 Développement du Serveur RESTful

5.1.1 Choix technologiques

Le serveur RESTful sera développé en utilisant :

- **Spring Boot** (Java) ou **Jersey** pour gérer les requêtes REST.
- **JPA/Hibernate** pour l'accès à la base de données.
- **JSON** comme format d'échange de données.

5.1.2 Endpoints à implémenter

Le serveur RESTful doit exposer toutes les fonctionnalités essentielles. Par exemple :

- **Authentification** :
  - `POST /auth/login` : Vérifie l'identifiant et le mot de passe de l'utilisateur.
- **Gestion des emails** :
  - `POST /emails/send` : Envoie un nouvel email.
  - `GET /emails/inbox/{userId}` : Récupère la liste des emails d'un utilisateur.
  - `DELETE /emails/{emailId}` : Supprime un email spécifique.
- **Gestion des utilisateurs** :

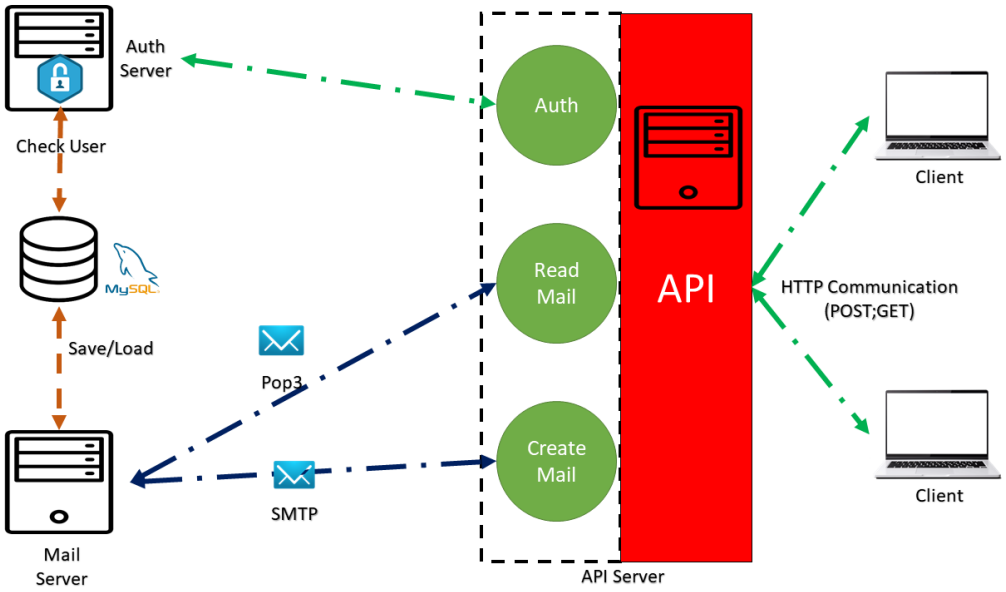

FIGURE 5 – Serveur RESTful

  - `POST /users/register` : Création d'un nouvel utilisateur.
  - `DELETE /users/{userId}` : Suppression d'un compte utilisateur.

5.2 Sécurité et Gestion des Accès

Pour sécuriser l'API, il est recommandé d'intégrer :

- **JWT (JSON Web Token)** pour l'authentification et l'autorisation des utilisateurs.
- **Contrôles d'accès basés sur les rôles** pour limiter certaines actions aux administrateurs (ex : suppression de comptes).

5.3 Intégration avec les Serveurs Existants

Le serveur RESTful agit comme une passerelle entre les clients et les serveurs SMTP, POP3 et d'authentification :

- Il envoie les requêtes d'authentification au **serveur d'authentification**.
- Il transmet les demandes d'envoi d'email au **serveur SMTP**.
- Il interroge le **serveur POP3** pour récupérer les emails stockés en base de données.

5.4 Tests et Validation

Les tests permettront de s'assurer que l'API fonctionne correctement et respecte les standards REST :

- **Tests unitaires** : Vérifier le bon fonctionnement des endpoints individuellement.
- **Tests d'intégration** : S'assurer que l'API communique correctement avec les serveurs SMTP, POP3 et d'authentification.
- **Tests avec Postman/cURL** :
  - Vérifier l'authentification des utilisateurs.
  - Tester l'envoi et la réception d'emails via des requêtes HTTP.
  - Vérifier la gestion des erreurs (ex : tentative d'accès non autorisé).
- **Scénarios de charge** : Tester les performances du serveur en simulant un grand nombre de requêtes concurrentes.

Dans le rapport, il faut :

- Expliquer le rôle et l'importance du serveur RESTful.
- Décrire les endpoints implémentés et leur fonctionnement.
- Présenter les résultats des tests avec des captures d'écran et des logs.
- Analyser les performances et proposer d'éventuelles améliorations.